Multimodal Manufacturing Creator

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	SRASHTI SEN	EN22CS301983
02	SOUMYA SHRIVASTAVA	EN22CS301978
03	TANISHKA RAGHUWANSHI	EN22CS3011019
04	TARUN PATIDAR	EN22CS3011034
05	VIDHI PRAJAPATI	EN22CS3011076

Group Name: Group 12D9

Project Number: GAI-12

Industry Mentor Name: Mr. Yash

University Mentor Name: Prof. Vineeta Rathore

Academic Year:2025-2026

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

1.1 Scope of the Document

The proposed system is a **Multimodal Generative AI Application** designed for **manufacturing concept visualization**. It integrates **Large Language Models (LLMs)** and **Text-to-Image Generative Models** to convert textual inputs into both descriptive explanations and visual representations.

This system helps users understand complex manufacturing concepts by generating:

- Detailed textual descriptions
- High-quality prototype images

The scope of this document includes:

- High-level architecture of the system
- Design of application components
- Data flow and processing
- API integrations
- Security and performance aspects

It does **not include low-level coding details**, but focuses on system-level understanding.

1.2 Intended Audience

This document is intended for:

- Software developers working on AI applications
- System architects designing scalable systems
- Academic evaluators and project reviewers
- Students studying Generative AI

1.3 System Overview

The proposed system is a **Multimodal Generative AI Application** that converts a user-provided text prompt into both a descriptive explanation and a visual representation of a manufacturing concept.

The workflow begins when the user enters a prompt through the Streamlit-based user interface. This input is then processed by two main components:

- **LLM (Text Generation Module):** The prompt is sent to a Large Language Model, which generates a detailed explanation of the manufacturing concept, including its working, components, and use cases.
- **Image Generation Model:** The same prompt is also passed to an image generation model (such as Stable Diffusion), which creates a visual prototype representing the concept.
- **UI Layer (Streamlit):** The generated text and image are displayed on the user interface in a clear and structured format, allowing users to easily understand and visualize the concept.

Overall, the system combines text and image generation to provide a comprehensive, multimodal output for better understanding and presentation of manufacturing ideas.

2. System Design

2.1 Application Design

The application is designed using a layered and modular architecture, which separates the system into different functional layers. This improves maintainability, scalability, and integration with external AI services.

1. Presentation Layer (User Interface)

The presentation layer is developed using Streamlit and serves as the interaction point for users. It provides:

• Input field for entering manufacturing-related prompts

• Display section for generated text explanations

• Image rendering area for visual outputs

This layer ensures a simple and user-friendly interface for interacting with the system.

2. **Application Layer (Backend Logic)**

The application layer acts as the core controller of the system. It is responsible for:

• Receiving and validating user input

• Managing the workflow of the system

• Preparing and sending API requests

• Collecting and formatting outputs

It ensures smooth communication between the UI and AI services.

---

3. AI Processing Layer

This layer contains the intelligence of the system using external APIs:

• LLM API (Text Generation):

Generates detailed textual explanations based on the input prompt

• Image Generation API:

Converts the same text prompt into a visual prototype or image

Both APIs are used to produce multimodal outputs.

---

4. API Integration Layer

This layer handles communication with external AI services:

• Sends requests to LLM and Image APIs

• Receives responses from APIs

• Manages API keys and request handling

This ensures reliable and efficient interaction with AI models.

---

5. Data Handling Layer

This layer handles temporary data during execution:

• Processes user input and API responses

• Does not use permanent storage or database

• Keeps the system lightweight and stateless

6. Integration Mechanism

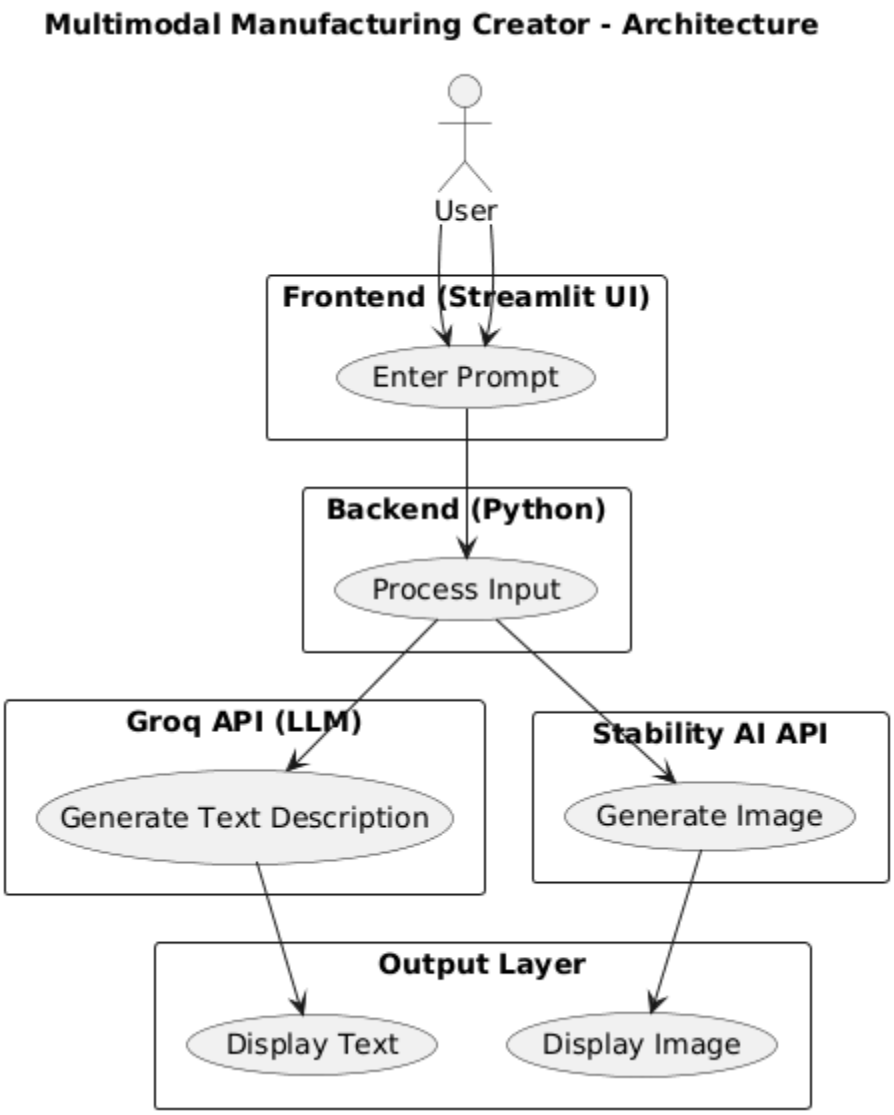
All layers are connected through well-defined interactions:

• UI communicates with backend via Streamlit functions

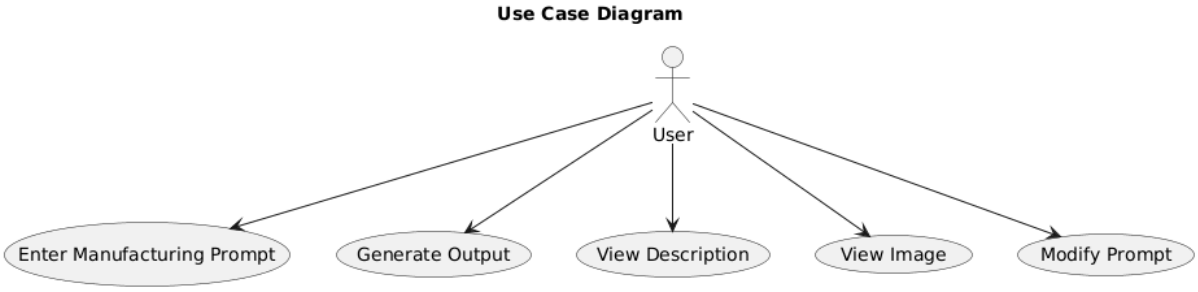
• Backend interacts with AI services using APIs

• Outputs are processed and returned to UI

_____Overall, the layered architecture ensures modularity, efficient processing, and easy scalability for future enhancements such as database integration or advanced AI models.



ARCHITECTURE DIAGRAM



USE CASE DIAGRAM

2.2 Process Flow

The process flow of the Multimodal GenAI Manufacturing Visualization System describes how a user's input is transformed into both textual and visual outputs using AI APIs.

Step 1: User Input

• The process begins when the user enters a manufacturing-related prompt in the Streamlit UI

• Example: "Design an automated robotic assembly line"

Step 2: Input Validation and Preprocessing

• The backend receives the input and performs:

- Validation (checks for non-empty input)

- Basic preprocessing (cleaning and formatting text)

• Ensures the input is suitable for API processing

Step 3: Request Handling by Backend

• The backend controller acts as the central coordinator

• It prepares API requests for:

- LLM API (text generation)
- Image Generation API

Step 4: Text Generation (LLM API)

• The prompt is sent to the LLM API (e.g., Groq)

• The model:

- Understands the context
- Expands the concept
- Generates structured explanation including components, working, and applications

---

Step 5: Image Generation (Image API)

• The same prompt is sent to the Image Generation API (e.g., Stability AI)

• The model:

- Interprets the prompt visually
- Generates a realistic or conceptual image

• Output is a prototype visualization

Step 6: Sequential Processing

• Text and image generation are processed sequentially

• This ensures stable execution and proper response handling

Step 7: Output Collection and Integration

• Backend collects:

- Generated text from LLM API

- Generated image from Image API

• Combines both into a unified response

Step 8: Response Delivery

• The combined output is sent back to the Streamlit UI

• UI displays:

- Text explanation

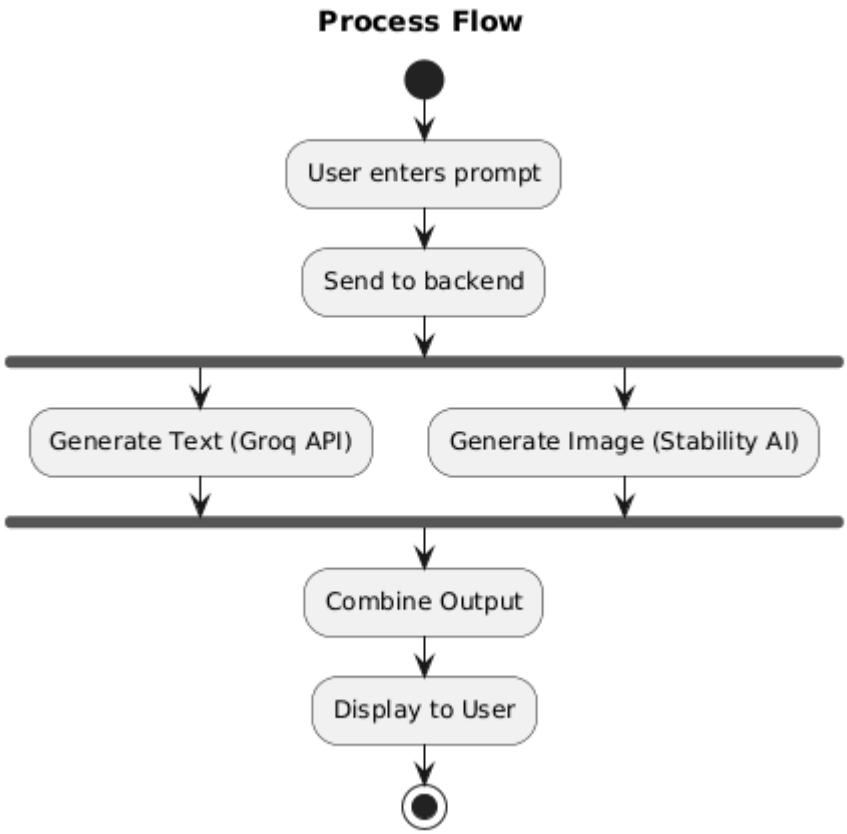
- Generated image

Step 9: Result Presentation

• The user views both outputs together

• This provides a clear multimodal understanding of the manufacturing concept

Overall, the process ensures smooth flow from input to output using API-based AI processing.

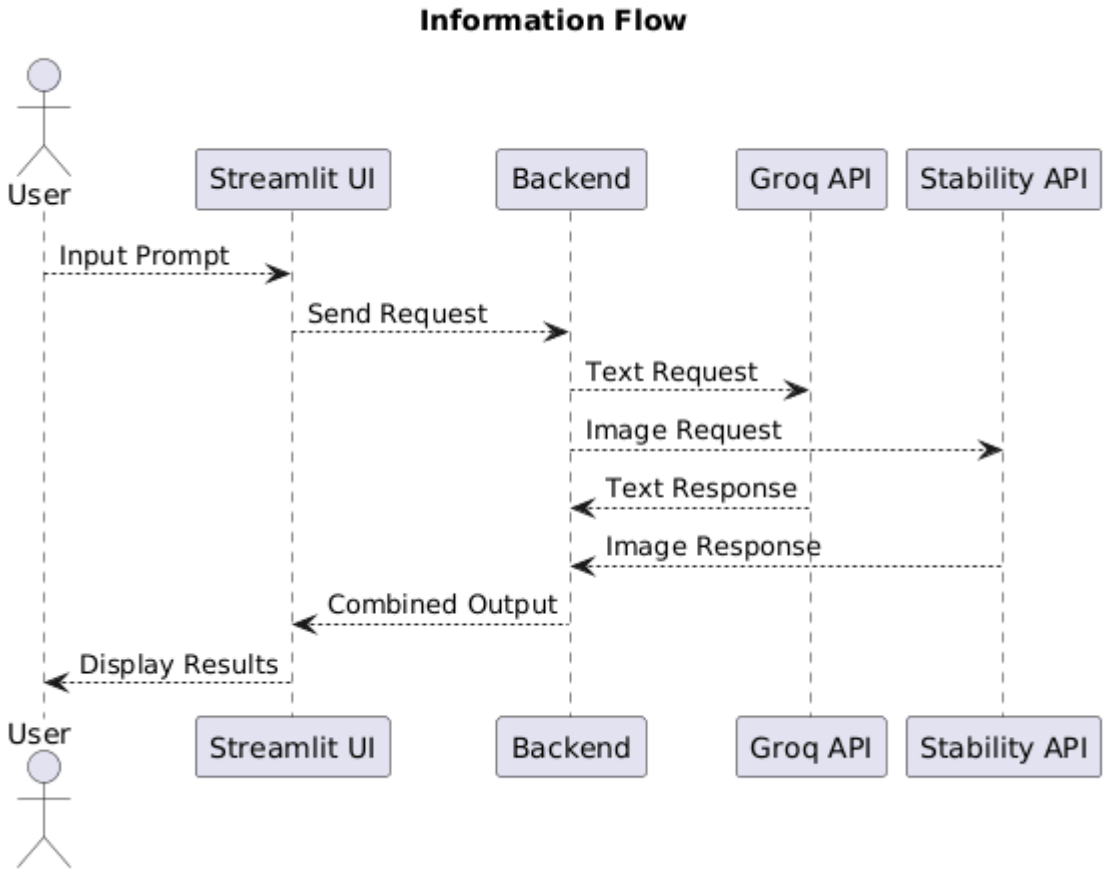


2.3 Information Flow

The system follows a structured data flow:

- **User Input:** User enters prompt in the Streamlit UI
- **Frontend to Backend:** UI sends input to backend controller
- **Input Processing:** Backend validates and processes the input
- **Model Invocation:** Backend forwards input to LLM and Image Model
-
- **Data Retrieval:** LLM retrieves context from ChromaDB (if required)
- **Output Generation:** AI modules return text and image outputs
- **Response Display:** Backend sends final output to UI for display

This ensures a smooth and efficient data pipeline.



2.4 Components Design

The component design of the Multimodal GenAI Manufacturing Visualization System defines the major functional units of the system and their interactions. The system is divided into modular components to ensure scalability, maintainability, and efficient processing.

1. Streamlit UI (Presentation Component)

• Acts as the user interface of the system

• Accepts manufacturing-related prompts from users

• Displays generated text explanations and images

• Provides an interactive and simple user experience

---

2. Backend Controller (Application Component)

• Acts as the central processing unit of the system

• Handles user requests and manages data flow

• Sends prompts to AI APIs (LLM + Image Generation)

• Integrates outputs into a unified response

---

3. LLM Module (Text Generation Component)

• Uses external LLM API (Groq / similar)

• Processes user input and generates structured explanations

• Produces outputs like:

- Concept description
- Components
- Working mechanism
- Applications

---

4. Image Generation Module (Visual Component)

• Uses external image generation API (Stability AI)

• Converts text prompts into visual representations

• Generates realistic prototype images

---

5. API Integration Layer

• Handles communication with external AI services

• Sends requests and receives responses from APIs

• Ensures smooth data exchange between backend and AI models

---

6. Component Interaction

• The UI sends user input to the Backend Controller

• The backend forwards the input to both:

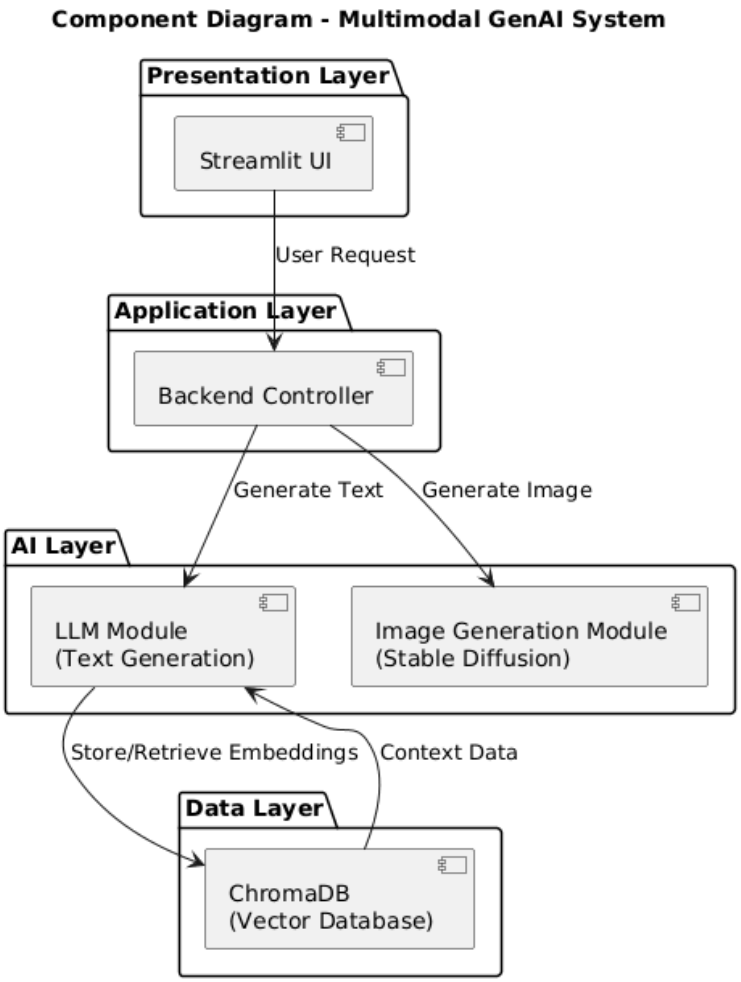
- LLM API (for text)
- Image Generation API (for visuals)

• Both APIs return outputs to the backend

• The backend combines text and image

• Final output is displayed on the UI

Overall, the component design ensures modularity, efficient processing, and easy scalability for future enhancements.



2.5 Key Design Considerations

- **Scalability:**
  The system is designed to handle multiple users and can be extended to cloud-based deployment for higher load handling.
- **Modularity:**
  Each component (UI, backend, LLM, image model, database) is independent, allowing easy updates or replacement without affecting the entire system.

- **Performance Efficiency:**
  Parallel processing of text and image generation is used to reduce response time and improve user experience.
- **Flexibility:**
  The architecture supports integration of different AI models or APIs, making the system adaptable to future technologies.
- **Usability:**
  A simple and interactive Streamlit interface ensures ease of use for both technical and non-technical users.

2.6 API Catalogue

The system uses a combination of external APIs and internal processing to generate multimodal outputs (text + image).

---

1. LLM API (Text Generation API)

• Purpose: Generate detailed textual explanations from user prompts

• Type: External API (Groq API / LLM API)

• Input: User prompt (manufacturing-related text)

• Output: Structured explanation (components, working, applications)

---

2. Image Generation API

• Purpose: Generate visual prototypes from text input

• Type: External API (Stability AI API)

• Input: Text prompt

• Output: AI-generated image

---

3. Backend Processing (Application Logic)

• Purpose: Handle request flow and integrate outputs

• Type: Internal (Python-based logic)

• Input: User input from UI

• Output: Combined text + image response

---

4. UI Interaction (Streamlit Interface)

• Purpose: Accept user input and display results

• Type: Internal Interface Layer

• Input: User prompt

• Output: Rendered text and image

Overall, these components work together to ensure smooth input processing, API communication, and generation of multimodal outputs.

3. Data Design

3.1 Data Model

The system manages different types of data generated during processing:

- **Prompt Data:**
  The input provided by the user describing a manufacturing concept.
- **Generated Text Data:**
  The explanation produced by the LLM, which includes detailed descriptions, working principles, and use cases.
- **Generated Image Data:**
  The visual output generated by the diffusion model, stored as an image file or URL.
- **Metadata:**
  Additional information such as timestamp, session ID, processing time, and model details.
- **Embedding Data:**
  Vector representations of the prompt or text stored in ChromaDB for similarity-based retrieval.

These data elements are logically connected, where a single prompt is associated with its corresponding text and image outputs along with metadata.

3.2 Data Access Mechanism

- All data access operations are controlled by the **backend controller**
- The system uses **ChromaDB** for storing and retrieving embedding vectors
- Retrieval is performed using **similarity search**, enabling context-aware responses
- Read and write operations are optimized to ensure fast data access
- API-based communication is used between backend and database

3.3 Data Retention Policies

The system follows flexible data retention policies:

- **Short-Term Storage:**
  - Temporary storage for testing or session-based use
- **Long-Term Storage:**
  - Documents stored for repeated querying

- **Cleanup Mechanism:**

- Old or unused data can be removed periodically
- Helps in optimizing storage and performance

- **User-Controlled Deletion:**
  - Option to delete uploaded documents (future enhancement)

3.4 Data Migration

- The system is designed to support migration from local storage to cloud platforms such as AWS or GCP
- Data can be exported in structured formats (JSON, CSV) and imported into other storage systems
- Migration ensures scalability when the number of users or data increases
- Backup and restore mechanisms can be implemented to prevent data loss during migration
- Future enhancements may include automated migration and distributed storage support

4. Interfaces

The system interacts with multiple internal and external components through well-defined interfaces to ensure smooth communication, modularity, and scalability.

1. **User Interface (UI)**
   - **Web/Frontend Interface:**
     - Allows users to input textual prompts describing manufacturing concepts.
     - Provides options to select output type: textual narrative, visual representation, or both.
     - Displays generated content in a structured layout (e.g., text on the left, images on the right).
     - Supports download/export of outputs in PDF, image, or text formats.
   - **Interaction Mode:** RESTful API calls from frontend to backend.
2. **Backend Interfaces**
   - **API Gateway / Controller Layer:**

- Receives requests from the frontend and routes them to appropriate services.

        - Validates input prompts and user metadata.
    - **Text Generation Service (LLM Interface):**
        - Communicates with the chosen LLM via API (e.g., OpenAI GPT or other hosted LLM).
        - Sends textual prompts and receives narrative outputs in JSON format.
    - **Image Generation Service (Text-to-Image Model Interface):**
        - Interfaces with image generation models (e.g., Stable Diffusion, DALL·E) via API.
        - Accepts structured prompts, parameters (resolution, style), and returns high-fidelity images in standard formats (PNG/JPEG).
    - **Data Storage Interface:**
        - Stores generated text and images in a persistent database or object storage.
        - Supports retrieval APIs for cached or historical content.

3. **External System Interfaces**
    - **Authentication/Authorization Interface:**
        - Integrates with OAuth or Single Sign-On (SSO) for user verification.
    - **Analytics / Logging Interface:**
        - Sends usage logs, prompt details, and system metrics to monitoring tools (e.g., ELK stack, Prometheus).

5. State and Session Management

- **Session Tracking:** Use JWT or session tokens to authenticate users and maintain their active session, with expiration and optional refresh tokens for continuity.
- **State Preservation:** Store current prompts, selected parameters, and generated outputs temporarily in frontend (React state) and backend (cache) to ensure smooth multimodal workflow.
- **Context & Security:** Preserve prompt-output context across requests for consistency in text and image generation, while securing session data with HTTPS and isolated user caches.

6. Caching

- **Prompt & Partial Caching:** Store mappings of prompts to generated outputs (text/images) and cache intermediate results (like LLM narratives) to reduce redundant API calls and speed up repeated or similar requests.
- **Expiration & Eviction:** Apply TTL (Time-to-Live) and eviction policies like LRU to manage cache size and remove old or unaccessed content efficiently.
- **Hybrid & CDN Strategy:** Use a combination of in-memory cache and persistent storage, and deliver high-resolution images via CDN to ensure fast retrieval, high availability, and improved user experience.

7. Non-Functional Requirements

7.1 Security Aspects

- **API Key Security:**
  API keys are stored securely using environment variables and are not exposed in the code.
- **Secure Communication:**
  All data transmission is done over HTTPS to ensure safe communication between components.
- **Input Validation:**
  User inputs are validated to prevent malicious or invalid data from affecting the system.
- **Data Privacy:**
  User data is not permanently stored, ensuring confidentiality and protection of information.

7.2 Performance Aspects

• **Moderate Response Time:**
The system generates text and images within a reasonable time depending on API response and network latency.

• **Sequential Processing:**
Text (LLM) and image generation are processed one after another to ensure stability and proper output handling.

• Efficient API Usage:
The system uses optimized API calls to generate outputs without unnecessary repeated requests.

• Lightweight Architecture:

Since the system is built using Streamlit and external APIs, it does not require heavy local computation.

• Scalability:
The system can be scaled easily by integrating faster APIs or deploying on cloud infrastructure.

8. References

- • Groq API Documentation – https://console.groq.com/docs
- • Stability AI Documentation – https://platform.stability.ai/docs
- • Streamlit Documentation – https://docs.streamlit.io
- • Python Requests Library Documentation – https://docs.python-requests.org
- • Pillow (PIL) Documentation – https://pillow.readthedocs.io